

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	Hi! I am Rohan Unbeg, an open-source enthusiast. I started contributing to Oppia a few months ago, and it quickly became a great learning experience. Oppia's welcoming community has helped me improve my debugging and communication skills significantly. I want to do a GSoC project here because I deeply connect with Oppia's mission of providing free, accessible education to everyone. Through my recent contributions to the translation infrastructure, I've seen firsthand how important accurate translations are for non-English speakers. Knowing that my work on this project will help translate more of Oppia's learning materials and directly impact learners globally is incredibly fulfilling, making this the perfect opportunity for me.	
What do you hope to learn/achieve during GSoC?	I'd like to learn how to work on a large-scale full-stack feature with datastores. Specifically, I hope to learn how to design extensible domain-layer architectures that can gracefully accommodate new entity types. I also want to gain practical experience writing production-grade Apache Beam jobs for data migrations. Beyond technical skills, I hope to become a better communicator, planner, and team collaborator by working closely with my mentors.	
Which Oppia teams have you collaborated with, and what have you done on those teams?	CORE	Participated in debugging doc reviews, provided feedback on PRs from other contributors, engaged in GitHub Discussions to help clarify translation infrastructure issues, and reported weekly progress updates during team syncs. (Coding contributions are listed in the Required Skills section below.)
Contact information	Email/Google Chat: rohanunbeg0918@gmail.com Phone: +91 8432216628	
Preferred method of communication	Email, Google Chat	
Social media handles (optional)	LinkedIn: rohanunbeg	
Which timezone will you primarily be in during the summer?	Indian Standard Time(IST) or GMT + 5:30	
(If you are a student) When are your school holidays?	My end-semester exams will finish by May 25. After that, I will have my summer break until August 14.	

What other obligations might you need to work around during the summer?	I have no other internships, jobs, or college classes during the GSoC coding period.
Planned time commitment	I will dedicate 35–40 hours per week during my summer break, and scale down to a maximum of 15 hours/week while my college is in session. Detailed Overview (April 30 – Sept 2): • Apr 30 – May 24 (Final Exams / Greenlight Period): 12–15 hours/week (2–3 hours/day). I will submit 3 early PRs (1.1, 1.2, 1.9) during this period while finishing exams and participating in Community Bonding. • May 25 – Aug 14 (Summer Break): 35–40 hours/week (6–7 hours/day). I have no classes or commute, making GSoC my absolute full-time focus for Milestone 1 and the bulk of Milestone 2. Critical note: Milestone 2's hard deadline is Aug 19, so all coding-heavy PRs are planned to be submitted by Aug 15 before college resumes. • Aug 15 – Sept 2 (College Resumes): 12–15 hours/week (2–3 hours/day). By this point, only PR 2.12 (documentation) and 2.13 (legacy code cleanup) remain. Testing/revision work only.

Project Timeframe

Note: Oppia will only be offering a single GSoC coding period timeframe this year, starting on **May 25**. All work for Milestone 1 must be completed and submitted by **Jun 26** for internal feedback (with any revisions due by **Jul 10**), and all work for Milestone 2 must be completed and submitted by **Aug 19** for internal feedback (with any revisions due by **Sep 2**). We will not be able to extend these deadlines.

Coding period	• I will adhere to the above deadlines.
----------------------	---

Communication Channels

Note: The Oppia team places a high emphasis on communication, and we have found that daily contact between contributors and mentors is important for helping keep projects on track. This is why we ask that contributors send short daily updates to the team group chat explaining what they have done, where they are stuck, and what they plan to do next.

I can commit to sending daily updates to the team group chat, each day I work during the GSoC period.	• Yes
In addition to the above: how often,	Daily updates on the mail, weekly syncs with the CORE Team

and through which channel(s), do you plan on communicating with your mentor?	and mentor via google meet and other communication via google chat
--	--

Contributor Briefing

Note: A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 1.	• Yes
--	---

Section 2: About Your Project

Project Details

Project title (should match the one on the Project Ideas list)	Extend translation infrastructure to exploration metadata and skills
Project size (should match the options on the Project Ideas list)	Large (~350 hours)
Why did you choose this project?	I chose this project because it perfectly aligns with the contributions I have been making to Oppia over the past few months. The GSoC project description explicitly states: "We've also had a number of issues with keeping translation counts up-to-date (see #21878), so you should address how you plan to avoid similar issues in your project." I am the contributor who diagnosed and fixed that exact 2+ year-old bug (PR #24589). I also wrote a Beam job to audit translation count mismatches (PR #25086 for #22094). Through debugging these issues, I became deeply familiar with opportunity_services.py, suggestion_services.py, EntityTranslationsModel, and the exact functions this project generalizes. Extending this infrastructure to support exploration metadata and skills is the natural next step - and I am uniquely positioned to do it correctly, having already identified and fixed the pitfalls. Additionally, the foundational work for the new translation opportunity structure (#22793) was started by Hasitha Kaushan (chris7716), who implemented the model, domain object, and a content-ID lookup method. However, the services PR (#22974) was closed due to bandwidth constraints, and the remaining 15+ milestones were never started. The project was recently re-scoped as #24933 to extend this structure to all user-generated content. I am the right person to pick this up because I have worked closely with Hasitha

	as my primary reviewer and understand both the existing code and the gaps that need to be filled.
--	---

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc .	Debugging Doc for #21878 - Identified the root cause of a 2+ year old translation count validation error. Created a systematic debugging doc with hypothesis, reproduction steps (including video proof of the race condition), and validated fix approach. GitHub Discussion #24660 .
I can debug and fix CI failures / flakes.	PR #25037 - Diagnosed and resolved a random CI timeout flake. Identified that Puppeteer's <code>waitForNavigation</code> was timing out on heavy external domains (YouTube/Medium) due to limited GitHub CI runner bandwidth. Documented the full investigation, root cause, and targeted fix in a linked debugging doc.
WEB	
I can write Python code with unit tests.	#24589 - Added regression tests for translation count validation, including tests for duplicate suggestion scenarios. #24726 - Fixed saving translations to the current exploration version, with comprehensive backend tests. #24444 - Persisted "Not Applicable" misconception status in questions, with domain-layer tests. #25065
I can write TS + Angular code with unit tests.	#24463 - Sorted translation contribution cards and added verification tests, demonstrating attention to UI behavior and test coverage. #24046 - Allowed current date in certificate download with Angular component updates and unit tests. #24151 - Used href for footer links and added hover effect, with i18n and responsiveness considerations. #25463
I can write or modify acceptance tests.	#25774 - Implemented new Puppeteer acceptance tests for the question editor modal (Issue #8704). This issue had been open since 2020 and saw multiple closed/stuck attempts by prior contributors; I successfully implemented the full end-to-end flow, including state management for the commit modal and new Puppeteer utilities. Status: All 180+ checks passing; currently awaiting CUJ/workflow verification from the dev-workflow team. #25037 - Refactored the newsletter sign-up acceptance test to optimize

	test suite performance and reliability. Replaced brittle, network-dependent UI navigation (<i>clickAndWaitForNavigation</i>) with robust external URL validation, demonstrating a strong grasp of asynchronous testing logic in Puppeteer.
I can write or modify Beam jobs , with tests.	#25086 - Implemented <i>ValidateExplorationOpportunityCountsJob</i> Beam job with testing doc. The job reads <i>ExplorationOpportunitySummaryModel</i> entities, computes true translation counts from <i>EntityTranslationsModel</i>, and reports mismatches. <i>Testing doc</i>. Status: Merged #25327 - Implemented <i>AuditDuplicateTranslationSuggestionsJob</i> and <i>CleanupDuplicateTranslationSuggestionsJob</i> Beam jobs to identify and clean up duplicate translation suggestions. Status: Approved and successfully passed backup server testing will get merged soon.
I can figure out reproduction steps based on information given in the server logs.	https://github.com/oppia/oppia/pull/24589#issuecomment-3876578728

(Optional) Oppia Community References

Who you worked with	Context in which you worked with them
Hasitha Kaushan (@chris7716)	Primary reviewer on PRs #24589 and #25086 . Collaborated extensively on debugging the translation count validation issue. Hasitha authored the TDD for this project and implemented the foundational PRs (#22792 , #22816 , #22852 , #23283). His services PR (#22974) was closed due to bandwidth constraints; this proposal continues that work.
Sambhav Kaushik	Served as my onboarding mentor for the core team. Guided my early backend contributions and development workflow, including reviewing and approving my initial work on PR #24444 , which successfully implemented and tested the persistence of the "Not Applicable" misconception status in practice questions.
Nitish Kumar	Reporting my weekly progress, task updates, and blockers to him during our weekly syncs in his capacity as the current Core Team Lead.
Kevin Thomas	Reviewer on PR #25065 (fixing the contributor dashboard progress bug). He reviewed my logic for excluding restricted cards for non-reviewers, provided feedback on properly documenting new backend datastore fields (<i>reviewer_only_content_count</i>), and guided me through the backup server testing process.
Hitesh Tomar	Reviewer on PR #25327 (adding Beam jobs for duplicate translation suggestions). He reviewed my backend data migration logic, suggested code refactoring to use the <i>DATASTORE_UPDATES_ALLOWED</i> pattern, and oversaw the final backup server job runs before approving.

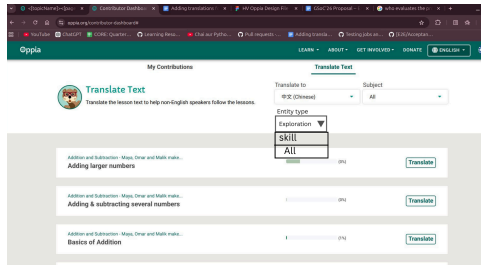
Section 3: Proposal Details

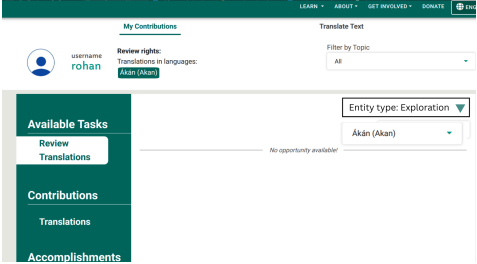
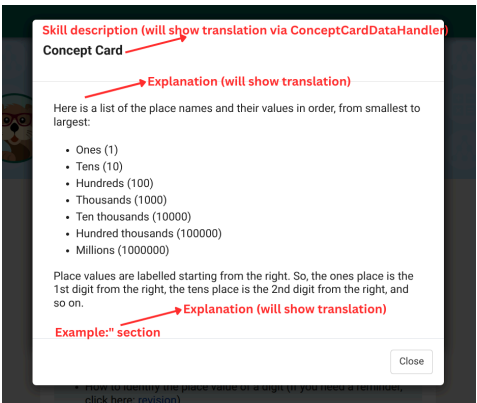
Problem Statement

Target Audience	Non-English speaking learners, ages 8-15, in target regions such as Brazil (Brazilian Portuguese speakers), the Middle East (Arabic speakers), and India (Hindi/regional language speakers). These learners cannot read English fluently and are discouraged by untranslated UI elements.
Core User Need	- As a non-English speaking learner, I need all text in Oppia's learning experience to appear in my language - not just the lesson content, but also lesson titles, skill descriptions, concept card explanations, topic names, and story titles - so that I can focus on learning without being blocked by unfamiliar English text. - Currently, even if a learner selects their preferred language, they still encounter English text in concept cards, lesson titles, topic metadata, and story navigation. - This project extends the existing translation infrastructure (which currently only covers exploration content) to also cover exploration metadata (titles, objectives, tags) and skill-related content (descriptions, concept card explanations).

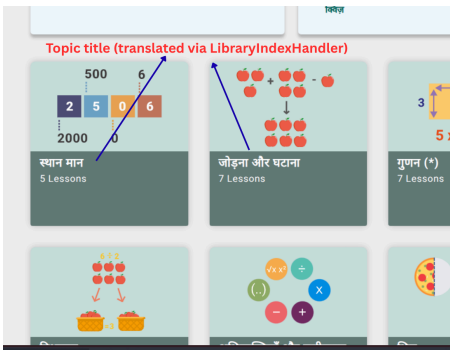
Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that ..."</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	Submitting translations for new entity types	As a translation contributor, I want to be able to submit translations for exploration metadata and skills, so that I can help translate Oppia's learning content beyond just lesson text.	- Open Contributor Dashboard → Translate tab. - Select entity type (exploration, skill) using the new entity-type-selector dropdown component. - Select topic and language. - View list of translation opportunities with progress indicators.	Figma: Concept Cards; TDD Section 1 Screenshot: 

			5. Click an opportunity to open the translation modal. 6. Translate the content and submit.	
2	Reviewing translations for all entity types	As a translation reviewer, I need to review and approve/reject translations for exploration metadata and skills, so that only high-quality translations reach learners.	1. Open Contributor Dashboard → Review tab. 2. Select entity type and language. 3. View submitted translations for review. 4. Compare original content with submitted translation. 5. Accept, edit+accept, or reject with feedback.	TDD User Story #2 Screenshot: 
3	Viewing translated concept cards	As a learner, I need to view concept cards (skill descriptions + explanations) in my selected language, so that I can understand prerequisite concepts without encountering English.	1. Play a lesson in a non-English language. 2. Click a concept card link. 3. View the concept card with skill description and explanation displayed in the learner's selected language.	TDD User Story #3; Concept Card Translation PRD Screenshot: 
4	Viewing translated lesson titles	As a learner, I want lesson titles to appear in my selected language so that I can navigate lessons without encountering English text.	1. Select a non-English site language. 2. Browse the Community Library or a topic page. 3. Lesson titles are displayed in the learner's language (if translations exist).	TDD User Stories #4, #6, #7 
5	Searching	As a learner, I want to	1. Select a non-English site	TDD User Story #5

	lessons by preferred language	search for lessons using terms in my preferred language so that I can find content without knowing the English title or tags.	language. - Go to the Community Library. - Type a search query in the preferred language. - Results include lessons with matching translated tags/titles.	Community Library search currently only indexes English tags. This project will extend elasticsearch indexing to include translated tags, enabling search in any language (User Story #5). Screenshot: 
6	Viewing translated exploration metadata in lesson info modal	As a learner, I want to see translated exploration titles and objectives in the lesson info modal, so I understand what the lesson covers before starting.	- Select a non-English language. - Open a topic page and click on a lesson tile's info icon. - The lesson info modal pops up - title and objective display in the selected language. - Falls back to English if no translation exists.	TDD Section: ExplorationSummariesHandler (lines 804-815) Technical detail: The modal is powered by LessonInformationCardModalComponent (lesson-information-card-modal.component.html). It fetches data from /explorationsummarieshandler/data. Per TDD: "There is no change to the existing response" - the JSON structure stays the same, but title, objective, and tags VALUES are populated from EntityTranslationsModel when translations exist. Current hacky behavior: The modal uses isHackyExpTitleTranslationDisplayed() with hardcoded i18n keys - only limited explorations have translations. After this change: All translated explorations will display properly. Screenshot:

				
7	Viewing translated metadata on classroom /topic pages	As a learner browsing a classroom page, I want exploration titles to display in my language so I can navigate lessons without English.	1. Select a non-English site language. 2. Browse the classroom page (/learn/math). 3. Exploration titles in stories display in selected language (via LibraryIndexHandler/topic page). 4. Falls back to English if there is no translation.	TDD User Stories #6, #7 Screenshot: 

Testing Plan

Critical user journeys

#	Goal ID	Goal / Sub-goal Title	Steps	Expectations
1	TS.CD.01	Translate exploration metadata and skill content Users: Curriculum Admin, Translation Contributor (Hindi).	-	-

		Setup: Create a topic with a story containing one exploration. Create a skill linked to the topic.		
1.1		Navigate to Contributor Dashboard	1. Click on the Profile dropdown avatar and select "Contributor Dashboard". 2. Navigate to the "Translate Text" tab. 3. Select entity type "Exploration" from the entity type dropdown. 4. Select the topic "Fractions" and language "Hindi".	Exploration metadata opportunities for "Fractions" appear. Progress bars show correct counts.
1.2		Translate exploration metadata	1. Click on the "Translate" button for an exploration (e.g., "Fair Shares"). 2. In the translation modal, translate the title, objective, and a tag. 3. Click "Save and close".	Toast "Submitted translation for review" appears. Content moves to "Awaiting review".
1.3		Translate skill content	1. Change entity type dropdown to "Skill". 2. Select the topic "Fractions". 3. Click on a skill opportunity (e.g., "unit fractions"). 4. Translate the skill description and explanation. 5. Click "Save and close".	Skill opportunities display separately from explorations. Progress reflects the submission.
2	TR.CD.01	Review translations across entity types Users: Translation	-	-

		Reviewer (Hindi). Setup: Same setup as TS.CD.01 plus submitted translations from Goal 1.2		
2.1		Review exploration metadata	1. Navigate to Contributor Dashboard --> "Review Translations" tab. 2. Select entity type "Exploration" and language "Hindi". 3. Click "Review" on the submitted metadata translation. 4. Accept the translation.	The reviewer sees translations organized by entity type. Accepted count increments.
2.2		Review skill content	1. Switch entity type to "Skill". 2. Click "Review" on the submitted skill translation. 3. Reject it with the message "Inaccurate translation".	Rejected skill translation appears in contributor's "Rejected" list with feedback.
3	LO.CL.01	Search lessons in non-English language Users: Learner. Setup: Same setup as TS.CD.01 + TR.CD.01 (accepted translations exist).	-	-
3.1		Search for translated content	1. Set site language to "Hindi". 2. Navigate to the Community Lessons page.	Results include lessons with matching translated tags. Non-translated lessons are filtered out.

			3. Enter the translated tag or title in the search bar.	
4	LO.CL.02	View translated concept cards and lesson titles Users: Learner. Setup: Same setup as Goal 3.	-	-
4.1		Verify lesson title and metadata	1. Start the lesson in Hindi.	Lesson title and metadata (objective, tags) display in Hindi.
4.2		Verify concept card	1. Click on a concept card link within the lesson.	The concept card shows description and explanation in Hindi. Concept card renders translated HTML explanation correctly.
5	TR.CD.02	Edge Case: Unpublished entity Users: Curriculum Admin, Translation Contributor, Translation Reviewer. Setup: Contributor submits a translation for a skill.	-	-
5.1		Verify auto-rejection	1. As Admin, unpublish the topic containing the skill. 2. As Reviewer, check the Review tab for the skill.	Pending translation suggestions are auto-rejected and removed.
5.2		Verify republishing	1. As Admin, re-publish the topic. 2. As a Contributor, check the Translate tab.	Translation opportunity reappears with fresh counts via <code>add_new_translation_opportunities()</code> .

Technical Requirements

Endpoints adapted from [TDD Section: "Additions/Changes to Web Server Endpoint Contracts"](#). My analysis adds notes on what changes from the current implementation.

Additions/Changes to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	New / Existing	Description of the request/response contract (and, if applicable, how it's different from the previous one). For new endpoints, include which access control decorator (e.g. "can_play_exploration", "open_access", etc.) will be used.
1.	/opportunitieshandler2	GET	New (replaces /opportunitiessummaryhandler)	Returns paginated translation opportunities filtered by entity_type (exploration, skill or None for all), language_code, and topic_name. Uses TranslationOpportunityModel instead of ExplorationOpportunitySummaryModel. Response includes entity_type, content_count, translation_counts, translation_in_review_counts, entity_description, is_pinned, and currently_available_to_learners. Multi-topic handling: A skill can belong to multiple topics. When no topic filter is applied, a deterministically chosen topic_name (e.g., the lexicographically first topic_name by sorting the topic_ids) is returned so that topic assignments are consistent across page reloads; when a topic filter is enabled, the selected topic_name is returned. Decorator: open_access (same as current handler).
2.	/gettranslatablecontentshandler2	GET	New (replaces /gettranslatabletexthandler)	Returns translatable contents for any entity type. Adds entity_type param. Response includes content_format, content_type, content_id, and content_value for each translatable field. Uses get_translatable_contents_collection() on the entity's domain object. Decorator: can_submit_translation_suggestions.
3	/suggestion handler	POST	Existing (modified)	Currently accepts target_type = exploration only. Will also accept target_type = skill when suggestion_type = translate_content (Support for target_type=topic and target_type=story is deferred to future work.). No change to request schema - only expanded enum of valid target_type values.

4	/getreviewableopportunitieshandler v2	GET	New (replaces /reviewableopportunitieshandler)	Returns reviewable translation opportunities filtered by entity_type and language_code. Uses TranslationOpportunityModel. Only returns opportunities in languages where the user has reviewer rights. Decorator: can_review_translation_suggestions.
5	/getreviewablesuggestionsv2	GET	New (replaces /getcontributionandreviewhandler)	Returns suggestions for review. Adds entity_type and entity_id filtering in addition to existing language_code and suggestion_type filters. Decorator: can_review_translation_suggestions.
6	/suggestionactionhandler/<suggestion_type>/<suggestion_id>	PUT	Existing (modified)	Modified to accept translations for skill and exploration metadata, updating how it processes the accepted translation based on entity type.
7	/entity_translations_handler/...	GET	Existing (modified)	Updated to fetch translations for the new entity types from the database.
8	/libraryindexhandler	GET	Existing (modified)	Current behavior: Returns exploration summaries with English-only metadata. After change: If user's preferred_language_codes[0] is not English, look up EntityTranslationsModel for each exploration. Populates title, objective, and tags fields with translated values. Falls back to English for missing translations. Per TDD: "There is no change to the existing response" - same JSON structure, different field VALUES. Example response field change: "title": "Adding Fractions" --> "title": "भिन्नों का जोड़" (if Hindi translation exists) Decorator: @open_access (unchanged).

9	/pinned-opportunities	PUT	Existing (modified)	Modified the LessonsPinningHandler so that it accepts the new generic opportunity models and supports entity_type alongside the opportunity ID. Changes from previous: Payload handling is updated to parse and store the entity_type of the pinned opportunity. New request field: "entity_type": "exploration" or "entity_type": "skill"
10	/concept_card_handler/<selected_skill_ids>	GET	Existing (modified)	Note: The TDD documents this endpoint in its Section 2 (Implementation Approach) but not in the Section 1 endpoint table. I am listing it explicitly here for completeness. Changes from previous: Adds optional language_code query parameter. Example: /concept_card_handler/["skill_123"]?language_code=hi If language_code is set, fetches translations from EntityTranslationsModel and populates skill_description, explanation.html, and worked_examples with translated values. Falls back to English for missing translations. Decorator: @can_view_skills (unchanged).
11	/explorationsummarieshandler/data	GET	Existing (modified)	Used by: LessonInformationCardModalComponent (lesson info modal popup). Current behavior: Returns exploration summaries with English-only metadata. After change: If user's preferred language is not English, looks up EntityTranslationsModel for each exploration. Populates title, objective, and tags fields in the summaries array with translated values. Per TDD (lines 804-815): "There is no change to the existing response" - same JSON structure, VALUES change. Example: Before: {"summaries": [{"title": "Adding Fractions",

				"objective": "Learn to add fractions", ...}}} After (Hindi): {"summaries": [{"title": "भिन्नों का जोड़", "objective": "भिन्नों का जोड़ सीखें", ...}]} Decorator: @open_access (unchanged).
--	--	--	--	---

Endpoint	Current Response (English)	After Change (Hindi preference)
#1 /opportunitieshandler2 (GET)	Request Params: entity_type (e.g. "skill") language_code (e.g. "hi") topic_name (optional) cursor (optional)	<pre>{ "opportunities": [{ "id": "exploration.exp_123", "entity_type": "exploration", "content_count": 12, "translation_counts": { "hi": 5 } }], "next_cursor": "cm9oYW4=", "more": true }</pre>
#2 /gettranslatablecontentshandler2 (GET)	Request Params: entity_type (e.g. "skill") entity_id (e.g. "skill_123")	<pre>{ "version": 3, "translatable_contents": [{ "content_id": "explanation", "content_format": "html", "content_type": "content", "content_value": "Add these fractions." }] }</pre>
#3 /suggestionhandler (POST)	<pre>{ "suggestion_type": "translate_content", "target_type": "skill", "target_id": "skill_123", "change_cmd": { "cmd": "add_written_translation", "content_id": "explanation",</pre>	<pre>{ "suggestion_id": "suggestion.translate_content.skill.skill_123.hi.v1" }</pre>

	<pre> "language_code": "hi", "translation_html": "इकाई भिन्न की पहचान करें।", "data_format": "html" } } </pre>	
#4 /getreviewableopportunitieshandle rv2 (GET)	Request Params: entity_type (e.g. "skill") language_code (e.g. "hi")	<pre> { "opportunities": [{ "id": "skill.skill_123", "entity_type": "skill", "topic_name": "Fractions", "translation_counts": { "hi": 1 } }], "more": false } </pre>
#5 /getreviewablesuggestionsv2 (GET)	Request Params: entity_type, entity_id, language_code, suggestion_type	<pre> { "suggestions": [{ "suggestion_id": "suggestion.translate_content...", "target_type": "skill", "change_cmd": { "translation_html": "इकाई भिन्न की पहचान करें।" } }] } </pre>
#6 /suggestionaction handler/<type>/<id> (PUT)	<pre> { "action": "accept", "commit_message": "Accepting translation.", "review_message": "Great work!" } </pre>	<pre> { "suggestion_id": "suggestion.translate_content.skill.skill_123.hi.v1", "status": "accepted" } </pre>
#7 /entity_translation	<pre> { "entity_type": "skill", </pre>	<pre> { "entity_type": "skill", </pre>

s_handler/... (GET)	<pre> "language_code": "en", "translations": { "explanation": { "content_format": "html", "content_value": "Identify unit fractions." } } </pre>	<pre> "language_code": "hi", "translations": { "explanation": { "content_format": "html", "content_value": "इकाई भिन्न की पहचान करें।" } } </pre>
#8 /libraryindexhandl er (GET)	<pre> { "activity_summary_dicts_by_category": [{ "activity_summary_dicts": [{ "id": "exp_123", "title": "Adding Fractions", "objective": "Learn to add fractions", "tags": ["math", "fractions"], "category": "Mathematics", "language_code": "en", "..." }] }], "preferred_language_codes": ["hi"] } </pre>	<pre> { "activity_summary_dicts_by_category": [{ "activity_summary_dicts": [{ "id": "exp_123", "title": "भिन्नों का जोड़", "objective": "भिन्नों का जोड़ सीखें", "tags": ["गणित", "भिन्न"], "category": "Mathematics", "language_code": "en", "..." }] }], "preferred_language_codes": ["hi"] } </pre>
#9 /pinned-opportuni ties (PUT)	<pre> { "entity_type": "skill", "entity_id": "skill_123", "action": "pin" } </pre>	<pre> { "status": "pinned", "entity_type": "skill", "entity_id": "skill_123" } </pre>
#10 /concept_card_ha ndler/<skill_ids>?l	<pre> { "concept_card_dicts": [{ </pre>	<pre> { "concept_card_dicts": [{ </pre>

language_code=hi (GET)	<pre> "skill_id": "skill_123", "skill_description": "Adding fractions with like denominators", "explanation": { "content_id": "explanation", "html": "To add fractions..." }, "worked_examples": "[...]" }] } </pre>	<pre> "skill_id": "skill_123", "skill_description": "समान हर वाले भिन्नों का जोड़", "explanation": { "content_id": "explanation", "html": "समान हर वाले भिन्नों को जोड़ने के लिए..." }, "worked_examples": "[...]" }] } </pre>
#11 /explorationsum marieshandler/ data (GET)	<pre> { "summaries": [{ "id": "exp_123", "title": "Adding Fractions", "objective": "Learn to add fractions", "tags": ["math", "fractions"], "category": "Mathematics", "language_code": "en", "ratings": "{...}", "num_views": 150, "..." }] } </pre>	<pre> { "summaries": [{ "id": "exp_123", "title": "भिन्नों का जोड़", "objective": "भिन्नों का जोड़ सीखें", "tags": ["गणित", "भिन्न"], "category": "Mathematics", "language_code": "en", "ratings": "{...}", "num_views": 150, "..." }] } </pre>

Note: Per TDD, "There is no change to the existing response" - same JSON structure, only **title**, **objective**, **tags** field VALUES are populated from EntityTranslationsModel when translations exist. Falls back to English for missing translations.

New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	Description of why the new call is needed, or why the changes to an existing call are needed
1	/opportunitieshandler/2	GET	The Contributor Dashboard frontend calls this with a new entity_type dropdown value. Replaces current call to

			/opportunitiessummaryhandler.
2	/gettranslatablecontentshandlerv2	GET	The translation modal calls this with entity_type + entity_id to fetch translatable contents for any entity. Replaces current call to /gettranslatabletexthandler.
3	/suggestionhandler	POST	The translation modal calls this to submit the user's translation suggestion. The payload will now include the generalized target_type (e.g., skill).
4	/getreviewableopportunitieshandlerv2	GET	The Contributor Dashboard's Review tab calls this to fetch a list of entities that have pending translations to review, filtered by the selected entity_type. Replaces /reviewableopportunitieshandler.
5	/getreviewablesuggestionsv2	GET	When a reviewer clicks an opportunity, the translation review modal calls this to fetch the actual suggested translation strings to review. Replaces /getcontributionandreviewhandler.
6	/suggestionactionhandler	PUT	The translation review modal calls this to accept, reject, or edit+accept the suggestion. Must be updated on the frontend to gracefully handle the generic opportunity contexts.
7	/pinned-opportunities	PUT	Pin/Unpin actions on the Contributor Dashboard. The pinning API calls will be updated to send entity_type along with the opportunity ID so different entities don't conflict.
8	/concept_card_handler/<skill_id>?language_code=xx	GET	<i>The Concept Card component (ConceptCardDataHandler) will now pass the learner's current language. Enables backend to return translated skill content (addresses TDD gap).</i>
9	/entity_translation_handler/<entity_type>/<entity_id>/<version>/<language_code>	GET	<i>Learner-facing pages (concept card component, lesson player) call this to fetch translated content for a specific entity and language.</i>
10	/libraryindexhandler	GET	<i>The Community Library page calls this to load exploration summaries. Response now includes translated titles/objectives based on the user's site language.</i>
11	/explorationsummarieshandler	GET	<i>The Lesson Information Card Modal (LessonInformationCardModalComponent) calls this to display lesson info. Response now includes translated</i>

	r/data		<i>title/objective/tags based on user's preferred language.</i>
--	--------	--	---

Web Server Storage Model Additions/Changes

#	Datastore model	Description of changes	How will existing data be handled?
1.	TranslationOpportunityModel (new, already partially implemented)	A single generic model for all entity types. Fields: entity_type, entity_id, topic_ids (list), content_count, translation_counts (dict), incomplete_translation_language_codes (list), language_codes_with_assigned_reviewers (list). ID format: {entity_type}.{entity_id}.	Existing ExplorationOpportunitySummary Model data migrated via Beam job. CRITICAL: The job will recompute content_count to include metadata fields (title/objective/tags) and recompute translation_counts from EntityTranslationsModel to ensure the new completion criterion is accurate.
2.	PinnedOpportunityModel (existing, modified)	Add entity_type field to distinguish pinned exploration opportunities from pinned skill opportunities.	Existing pinned opportunities will have entity_type set to exploration via a Beam migration job.

UI Screens/Components

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	entity-type-selector	New dropdown in the Contributor Dashboard (both Translate and Review tabs) allowing users to filter by entity type (Exploration, Skill, All). Built to be extensible for future entity types (Topic, Story).	Yes (dropdown labels)	Figma	ARIA label, keyboard navigation, screen reader support for selection changes.

2.	oppia-translation-display	Deferred to future work (with Topic/Story support). TDD Section 4 describes this component with a BaseCard class hierarchy for unified translated content display. For GSoC scope (exploration metadata + skills), the learner-facing user stories are achieved by updating backend handlers to return translated field values directly - existing frontend components display this data. The full component architecture is better suited for future work when Topic/Story translations are added.	No (displays user-submitted translations)	TDD section 4	Alt text for any embedded images, proper heading hierarchy.
3	translated-concept-card	Modified concept card component. Now accepts a <code>language_code</code> prop and fetches translated content if available. Falls back to English for untranslated fields.	No (displays translations from datastore)	Concept Card PRD	Same A11y as the current concept card component.

Data Handling and Privacy

#	Type of data	Description	Why do we need to store this data?	Anonymized?	Can the user opt out?	Wipeout policy	Takeout policy
1.	Translation opportunity metadata	TranslationOpportunityModel: entity type, content counts, translation progress per language.	Required to power the Contributor Dashboard - showing which entities need translations and tracking progress.	Yes (no user data - only entity metadata and aggregate counts).	N/A (not user-specific data).	Model is deleted when the corresponding entity is deleted.	Not included (not user data).
2.	Pinned opportunity preferences	PinnedOpportunityModel: user ID + list of pinned opportunity IDs with entity type.	Allows contributors to bookmark specific translation tasks for easy access.	No (linked to user ID).	Yes (user can unpin).	Deleted with user account (existing behavior).	Included in takeout (existing behavior).

Connection to Existing Work

This project directly builds on and depends on the following existing work:

- [#21878](#) / [PR #24589](#): Fixed the root cause of translation counts exceeding content counts. This PR changes `update_translation_opportunity_with_accepted_suggestion` to recount translations from `EntityTranslationsModel` instead of blindly incrementing - the exact function that this project generalizes. The fix also adds duplicate suggestion prevention guards. The

recompute-from-source pattern introduced in this PR is the foundation of Decision 3 in this proposal.

- **#22094 / PR #25086**: Beam job to audit translation count mismatches. Validates that `ExplorationOpportunitySummaryModel.translation_counts` matches the actual translations in `EntityTranslationsModel`. The audit pattern is directly reusable for the backfill Beam jobs in this project (TDD PRs 1.15, 2.5).
- **Existing BaseTranslatableObject pattern ([translation_domain.py](#))**: Already implemented for explorations. This project extends it to Skill domain objects. (Topic and Story extensions are deferred to future work.)
- **Existing TranslationOpportunityModel ([gae_models.py](#))**: The generic model is already partially implemented with `entity_type`, `topic_ids`, and `translation_counts` fields.
- **#22793 (parent tracking issue)**: Hasitha's milestone plan for the new translation opportunity structure. PRs 1.1–1.3 and 1.6 are merged; PR 1.4/1.5 ([#22974](#)) was closed due to inactivity; PRs 1.7–1.19 were never started. This GSoC project picks up from where that effort stalled.
- **#24933 (new tracking issue)**: Opened by @U8NWXD to extend the translation opportunity structure to all user-generated content in the curated curriculum. This is the issue that directly corresponds to the GSoC project scope.

Other Requirements

- **Accessibility (A11y)**: All new UI elements (e.g., the entity-type selector dropdown in the Contributor Dashboard) must be accessible via keyboard and screen readers, maintaining Oppia's current accessibility standards with ARIA labels.
- **Internationalization (i18n)**: User-facing text strings introduced by this project (e.g., dropdown labels like "Exploration", "Skill") must be added to the global translation files for localization.
- **Backward Compatibility**: Existing `PinnedOpportunityModel` entities currently assume they represent explorations. The migration script must explicitly set their new `entity_type` field to `exploration` so that existing users' bookmarks do not break.

Section 3.2: HOW

Existing Status Quo

Oppia's contributor dashboard currently supports translating only the content inside exploration states (card content, hints, solutions, feedback, interaction customization args, and rule inputs).

Storage: The current translation infrastructure uses several models:

- **ExplorationOpportunitySummaryModel** (in [opportunity/gae_models.py](#)): Stores one record per exploration with `content_count`, `translation_counts` (dict: `language_code` → int), and `incomplete_translation_language_codes`. The model ID is the exploration ID - it cannot represent other entity types.
- **EntityTranslationsModel** (in [translation/gae_models.py](#)): Stores the actual translated content per entity-version-language tuple. Fields: `entity_id`, `entity_type`, `entity_version`, `language_code`, and `translations` (dict: `content_id` → `TranslatedContent`). Currently `entity_type` is limited to choices `[EXPLORATION, QUESTION]` - `SKILL` is not yet supported.

- **PinnedOpportunityModel** (in `user/gae_models.py`): Stores a user's pinned translation opportunities for quick access. Fields: `user_id`, `language_code`, `topic_id`, `opportunity_id`. Currently has no `entity_type` field - it assumes all pinned items are explorations.
- **GeneralSuggestionModel** (in `suggestion/gae_models.py`): Stores translation suggestions. Fields include `suggestion_type`, `target_type`, `target_id`, `target_version_at_submission`, `status`, `author_id`, `change_cmd`, and `language_code`. The `target_type` choices already include 'skill', but the translation suggestion flow currently only accepts 'exploration'.

Domain: `ExplorationOpportunitySummary` (in `opportunity_domain.py`) is the corresponding domain object. `Exploration` extends `BaseTranslatableObject`, but its `get_translatable_contents_collection()` (in `exp_domain.py`) only iterates over `self.states` - it excludes metadata fields (title, objective, tags). `Skill` does not extend `BaseTranslatableObject` at all.

Services: `opportunity_services.py` manages the lifecycle: `add_new_exploration_opportunities()` is called when a story is published; `update_translation_opportunity_with_accepted_suggestion()` increments `translation_counts[lang]` by 1 per accepted suggestion.

Controllers/Frontend: Controllers accept `target_type = exploration` only. The translation modal hardcodes `target_type: 'exploration'` and structures content as `state_names_to_content_id_mapping`. There is no entity-type selector.

What's missing: 1. Exploration metadata cannot be translated. 2. Skill content cannot be translated. 3. The opportunity model is exploration-specific. 4. Learner-facing pages show metadata in English even when translations exist.

What's already been built: Hasitha (chris7716) has implemented foundational pieces gated behind the `ENABLE_TRANSLATION_OPPORTUNITIES_WITH_NEW_OPP_MODELS` feature flag (currently DEV stage):

- `TranslationOpportunityModel` (in `gae_models.py`): Generic model with `entity_type`, `entity_id`, `topic_ids` (list), `content_count`, `translation_counts`. ID format: `{entity_type}.{entity_id}`. The model's `_pre_put_hook()` already validates `entity_type` against {'exploration', 'skill', 'topic', 'story', 'classroom'}, meaning skill support is architecturally prepared but not yet wired to services or controllers.
- `TranslationOpportunity` and `TranslationOpportunityCardInfo` domain objects (`opportunity_domain.py`).
- Basic opportunity services (closed [PR #22974](#) - needs revival as part of PR 1.3).
- Content-ID lookup method for exploration states ([PR #23283](#), merged).

Problem	Root Cause	Solution Approach
Exploration metadata (title, objective, tags) cannot be translated	<code>Exploration.get_translatable_contents_collection()</code> only iterates over states, excluding metadata fields	Extend method to include metadata with synthetic content IDs (<code>exploration_title</code> , <code>exploration_objective</code> , <code>tag_{name}</code>)
Skill content cannot be translated	Skill does not extend <code>BaseTranslatableObject</code> ; no <code>get_translatable_contents_collection()</code>	Make Skill extend <code>BaseTranslatableObject</code> ; implement method with <code>skill_description</code> + explanation content IDs

Opportunity model is exploration-specific	ExplorationOpportunitySummary Model uses exploration ID as model ID - cannot represent other entities	Migrate to generic TranslationOpportunityModel with entity_type field
EntityTranslationsModel doesn't support skills	TranslatableEntityType enum only has EXPLORATION and QUESTION	Add SKILL to the enum
Contributors cannot submit translation suggestions for new content types	SuggestionHandler only accepts target_type=exploration; TranslationModalComponent hardcodes exploration in payload	Extend SuggestionHandler.post() to accept target_type=skill; update TranslationModalComponent to pass correct entity_type; use __metadata__ sentinel for state_name in metadata suggestions
Reviewers cannot review translation suggestions for skills or metadata	GetReviewableOpportunitiesHandler only queries suggestions with target_type=exploration	Extend NewGetReviewableOpportunitiesHandlerV2 and NewGetReviewableSuggestionsV2 to filter by entity_type; update SuggestionActionHandler to dispatch accept logic based on entity_type
Learner-facing pages show English metadata even when translations exist	Handlers (LibraryIndexHandler, ConceptCardDataHandler) don't look up EntityTranslationsModel	Update handlers to fetch and serve translated content based on user's preferred language
Pinned translation opportunities don't distinguish entity types	PinnedOpportunityModel has no entity_type field - all pinned items are assumed to be explorations. Pinning a skill opportunity would conflict with exploration pins.	Add entity_type field to PinnedOpportunityModel (default 'exploration'). Update LessonsPinningHandler to accept entity_type in request payload. Backfill existing pinned records with entity_type='exploration' via Beam job (PR 1.10).

Entity Types and Content IDs Covered in This Project:

Entity Type	target_type	Content	content_id	Content Format
Exploration (metadata)	exploration	Title	exploration_title	unicode
Exploration (metadata)	exploration	Objective	exploration_objective	unicode
Exploration (metadata)	exploration	Tags	tag_{tagName} (e.g., tag_algebra)	unicode
Skill (concept card)	skill	Description	skill_description	unicode
Skill (concept card)	skill	Explanation	explanation (existing SubtitledHtml content_id)	html
Skill (concept card)	skill	Misconception Feedback	misconception_{id}_feedback	html

Out of Scope (Future Work):

Entity Type	Example Contents	Notes
topic	title, description, subtopic_title_{n}	Deferred per GSoC scope
story	title, description, story_node_title_{n}, story_node_description_{n}	Deferred per GSoC scope
subtopic_page (study guide)	section_heading_{n}, section_content_{n}	Deferred per GSoC scope

The solution extends Oppia's translation infrastructure in four areas:

1. **Extensible Backend Framework (Milestone 1):** Migrate the existing exploration translation workflow from `ExplorationOpportunitySummaryModel` to the generic `TranslationOpportunityModel`. Re-wire all service methods and controllers. A Beam job backfills the new model. Old model/endpoints retained behind the feature flag for rollback.
2. **New Entity Support (Milestone 2 - Backend):** Extend the framework to support exploration metadata (title, objective, tags) and skill content (description, concept card explanation). Make `Skill` a `BaseTranslatableObject`, extend `Exploration.get_translatable_contents_collection()` to include metadata, create opportunity lifecycle hooks for skills, update `SuggestionHandler` to accept `target_type=skill`, and update `SuggestionActionHandler` to dispatch based on `entity_type`.
3. **Contributor Dashboard Frontend (spans both milestones):** Add entity-type-selector dropdown component to both Translate and Review tabs. Update `TranslationModalComponent` to handle skill and metadata entities (fetch from `/gettranslatablecontentshandlerv2`, submit with correct `target_type`). Update `TranslationOpportunitiesComponent` to display skill opportunities. Update Backend API Services with feature-flag-aware routing to call v2 endpoints.
4. **Learner-Facing Display (Milestone 2):** Update learner-facing endpoints (`LibraryIndexHandler`, `ExplorationSummariesHandler`, `ConceptCardDataHandler`) to serve translated content when the user has selected a non-English language. Update `ConceptCardComponent` to pass `language_code` and display translated skill content.

Third-Party Libraries

No new third-party libraries are required. This project uses only existing Oppia dependencies (Angular, Apache Beam, Google Cloud Datastore).

"Service" Dependencies

Note: This implementation extends the existing Oppia backend architecture and introduces zero new external service dependencies.

Key High-Level and Architectural Decisions

Decision 1: How to make exploration metadata translatable

Problem: Currently, `Exploration.get_translatable_contents_collection()` only iterates over `self.states`. Metadata fields (`title`, `objective`, `tags`) are plain strings without `content_ids`, so they cannot be translated through the existing content-ID-based system.

We have considered the following alternatives:

- Add metadata fields to `get_translatable_contents_collection()` with synthetic content IDs. Assign content IDs `"exploration_title"`, `"exploration_objective"`, and value-based tag IDs like `"tag_{tagName}"` (e.g., `tag_algebra`, `tag_fractions`). This makes them part of the `BaseTranslatableObject` framework. The `content_count` automatically includes them.
- Store metadata translations in a separate system (e.g., a new `MetadataTranslationsModel` or `translatewiki`). This keeps the existing `BaseTranslatableObject` framework unchanged.
- Follow the TDD approach: Return metadata alongside state-level content using the existing `TranslatableContent` response format (a backend-to-frontend representation, not a new storage model), but track progress separately from the `content_count/translation_counts` system.

Among these, I believe **Alternative 1** is the best approach, because:

- It reuses the existing `BaseTranslatableObject` --> `TranslatableContentsCollection` --> `EntityTranslationsModel` pipeline with zero new models or abstractions.
- `content_count` and `translation_counts` automatically include metadata, giving accurate progress tracking without special-case code.
- The `get_translatable_contents_collection()` pattern is already used by `State`, `InteractionInstance`, `Hint`, `Solution`, `AnswerGroup`, and `Outcome`. Extending it to metadata follows an established pattern.
- The alternative of tracking metadata translations separately creates dual bookkeeping that risks the exact translation count mismatches we fixed in Issue #21878.

The above approaches are contrasted in detail in the following table:

Criteria	Weight	Alternative 1: Synthetic IDs (CHOSEN)	Alternative 2: New Model	Alternative 3: Separate Tracking
No new storage models	Critical	Uses existing <code>EntityTranslationsModel</code>	Requires <code>MetadataTranslationsModel</code> + migration	Uses existing model but separate tracking
Unified progress tracking	High	<code>content_count</code> includes metadata automatically	Creates dual bookkeeping (risk: #21878 recurrence)	Requires two progress indicators
Content ID collision prevention	High	Prefixed IDs (<code>exploration_title</code> , <code>tag_{name}</code>) never collide with state integer IDs	N/A	N/A

Follows existing patterns	Medium	Same pattern as State, Hint, Solution, AnswerGroup	New pattern to maintain	Hybrid approach
Implementation effort	Medium	Low (~2 days)	High (~1 week + migration)	Medium (~4 days)

Implementation detail: I will add a new `ContentType` enum value `METADATA` in `translation_domain.py` and assign the following fixed content IDs:

Metadata field	Content ID	Content format
title	exploration_title	unicode
objective	exploration_objective	unicode
tags[i]	tag_{tagName} (e.g. tag_algebra, tag_fractions)	unicode

These will be added in `Exploration.get_translatable_contents_collection()` before the existing state iteration loop.

Risks: 1. **Content ID collisions** - metadata IDs use the `exploration_` and `tag_` prefixes, which never collide with state-level integer-based IDs (`content_0`, `ca_choices_1`). The existing `validate_translatable_contents()` only checks numeric suffixes, so metadata IDs are naturally excluded.

2. **Tag changes** - value-based IDs (`tag_algebra`) ensure reordering doesn't invalidate translations. Removed tags have their translations deleted; added tags appear as new untranslated content.

3. **Content ID naming** - we use prefixed IDs (e.g., `exploration_title` instead of the TDD's bare `title`) to avoid any potential collision with state-level content IDs, which use unprefixed numeric patterns like `content_0`.

Decision 2: How to make Skill translatable

Problem: `Skill` does not extend `BaseTranslatableObject`. Its concept card (`SkillContents`) has an explanation field that uses `SubtitledHtml` (with a `content_id`), but the skill's description is a plain string. Misconception notes/feedback and rubric explanations are also plain HTML strings without content IDs.

We have considered the following alternatives:

- Make `Skill` extend `BaseTranslatableObject` and implement `get_translatable_contents_collection()` to include the description (with a synthetic content ID), delegate to `SkillContents` for the explanation field, and dynamically include misconception feedback.
- Only make `SkillContents` a `BaseTranslatableObject`, since the explanation already has a `content_id`. Add a separate mechanism for the description.

Among these, I believe **Alternative 1** is the best approach, because:

- It follows the same pattern used for Exploration - a top-level domain object that implements `get_translatable_contents_collection()`.
- The description field gets a synthetic content ID ("`skill_description`", format unicode), while the explanation content ID is already assigned via `SubtitledHtml`.
- `content_count` for a skill will dynamically be $2 + N$ (where N is the number of misconceptions) - clean, predictable, and fully extensible.

The above approaches are contrasted in detail in the following table:

Criteria	Alternative 1: Make Skill extend <code>BaseTranslatableObject</code>	Alternative 2: Only make SkillContents a <code>BaseTranslatableObject</code>
Consistency with Exploration	Yes. Top-level domain object implements <code>get_translatable_contents_collection()</code> .	No. <code>SkillContents</code> is translatable but description needs a separate mechanism.
Single translation path	Yes. One unified path for all skill content.	No. Two different translation paths for one entity.
Description translation	Handled via synthetic <code>skill_description</code> content ID.	Requires separate mechanism outside <code>BaseTranslatableObject</code> .
Extensibility	High. Future fields (misconceptions, rubrics) can be added to <code>get_translatable_contents_collection()</code> .	Low. Adding new fields requires building new mechanisms each time.

Worked Examples Scope Decision

Context: The TDD lists `WorkedExample` as translatable, but the codebase changed after GSoC 2024 cleanup (PRs #22902, #22883). `WorkedExample` was removed as a separate storage class in schema migration v4-->v5 (`skill_domain.py:1363`).

Criteria	Option A: Separate worked_example content IDs	Option B: Translate as part of explanation (CHOSEN)
Storage model	Requires reverting v4-->v5 migration to restore <code>WorkedExample</code> class	Uses current schema - worked examples are RTE components in explanation HTML
Content IDs	<code>worked_example_question_{n}</code> , <code>worked_example_explanation_{n}</code>	explanation (single <code>content_id</code> covers all embedded worked examples)

content_count for skills	2 + N (where N = number of worked examples)	2 (description + explanation-with-worked-examples)
Migration complexity	HIGH - must revert schema migration, update all skill models	NONE - works with current schema
Translation UX	Worked examples appear as separate translatable items	Worked examples translated inline within explanation HTML
Codebase alignment	Conflicts with GSoC 2024 cleanup that removed WorkedExample class	Aligns with current codebase structure
Scope fit	Outside GSoC scope - requires storage restructuring first	Within GSoC scope - extends existing BaseTranslatableObject pattern

Skill Content Scope: What Gets Translated?

Content Type	Learner-Facing?	Included in Scope?	Rationale	content_id Pattern
skill_description	Yes (displayed in concept cards)	YES	Core skill identifier shown to learners	skill_description
explanation	Yes (concept card explanation)	YES	Main learning content with embedded worked examples	explanation (existing SubtitledHtml content_id)
misconception.feedback	Yes (shown when learner has misconception)	YES	"The feedback given to learners when they have this misconception" (skill_domain.py:472)	misconception_{id}_feedback (e.g., misconception_0_feedback)
rubrics	NO (internal guidelines for question creators)	NO	"Create guidelines to help question creators make questions under a certain rubric" (rubrics-editor.component.html) - not shown to learners	N/A

Updated content_count formula for skills:

$$\text{content_count} = 2 + N$$

Where:

- **Base:** skill_description (1) + explanation (1) = 2
- **Plus:** misconception_{id}_feedback for each misconception (N)
- **Example:** A skill with 3 misconceptions has content_count = 5

Note: The TDD specifies max-length validations for translated metadata fields (e.g., exploration title <= 36 chars, skill description <= 100 chars). These validations will be implemented in the suggestion submission handler and translation modal as part of PRs 2.1 and 2.4.

Decision 3: How to handle translation counts accurately

Problem: Issue #21878 revealed a 2+ year old bug where translation_counts drifted from reality. Through my debugging (debugging doc), I traced the root cause to two sources: (1) update_translation_opportunity_with_accepted_suggestion() blindly incremented counts without checking for duplicates, and (2) race conditions between concurrent suggestion acceptances could double-count translations. My PR #24589 fixed this by replacing increment logic with a recompute-from-EntityTranslationsModel approach. **This exact pattern is the foundation for count handling in this project** - I have already shipped the fix, so extending it to skills and metadata is natural.

We have considered the following alternatives:

- Recompute counts from EntityTranslationsModel on each accepted suggestion (recompute-from-source).
- Keep increment-by-1 approach but add duplicate guards before incrementing.

Among these, I believe **Alternative 1** is the best approach, because:

- Counts can never exceed actual translations (fixes #21878).
- Idempotent - safe to retry. Already proven in merged PR #24589.

The above approaches are contrasted in detail in the following table:

Criteria	Alternative 1: Recompute from EntityTranslationsModel	Alternative 2: Increment-by-1 with duplicate guards
Count accuracy guarantee	Strong. Count always equals actual translations in datastore.	Weak. Race conditions can still cause drift between writes.
Idempotency	Yes. Re-running produces the same result.	No. Each retry increments again.
Performance cost	One extra datastore read per accepted suggestion.	No extra read, but requires pre-check queries for duplicates.
Proven in production	Yes. Pattern shipped in PR #24589 (LGTM'd, merged).	No. This is exactly what caused the 2+ year bug in #21878.

Decision: All count updates in the new system will recompute from the source of truth (`EntityTranslationsModel`) rather than incrementing, providing eventual consistency. In the unlikely case of two simultaneous acceptances, one count may be momentarily stale, but the next write recomputes correctly - and the count cannot exceed the actual translation count (which was the critical bug in [#21878](#)). The periodic validation Beam job provides a safety net for any drift. Specifically:

1. When a translation suggestion is accepted, `update_translation_opportunity_with_accepted_suggestion()` will fetch the `EntityTranslations` object for the entity and call `get_translation_count(language_code)` (inherited from `BaseTranslatableObject`) to get the true count.
2. The backfill Beam job will compute `content_count` from the entity's `get_translatable_contents_collection()` and `translation_counts` from `EntityTranslationsModel`, never from the old `ExplorationOpportunitySummaryModel`.
3. A validation Beam job (extending my existing PR [#25086](#)) will run periodically to catch any drift.

Decision 4: Dual-endpoint strategy during migration

Problem: We need to switch from old endpoints (`/opportunitiessummaryhandler`, `/gettranslatabletexthandler`) to new ones without breaking the existing workflow.

We have considered the following alternatives:

- Create v2 endpoints alongside old ones, gated behind a feature flag. Remove old endpoints after transition.
- Modify existing endpoints in-place to support new entity types directly.

Among these, I believe **Alternative 1** is the best approach, because:

- Safe rollback - revert flag to disable all new behavior.
- No breaking changes during migration.

The above approaches are contrasted in detail in the following table:

Criteria	Alternative 1: v2 endpoints with feature flag	Alternative 2: In-place endpoint modification
Rollback safety	High. Revert feature flag to disable all new behavior instantly.	None. Any bug breaks existing users immediately.
Code duplication	Temporary. Old and new coexist until cleanup PR (2.13).	None, but higher risk.
Migration path	Gradual. DEV --> TEST --> PROD with validation at each stage.	All-or-nothing deployment.
Future cleanup	PR 2.13 removes old endpoints and drops v2 suffixes.	N/A.

Decision: Following the TDD's approach, v2 endpoints (`/opportunitieshandlerv2`, `/gettranslatablecontentshandlerv2`, etc.) will be created alongside the old ones, using the v2 suffix convention per Sambhav's feedback. The frontend will call the new endpoints when `ENABLE_TRANSLATION_OPPORTUNITIES_WITH_NEW_OPP_MODELS` is enabled. After the feature flag is flipped to PROD and the transition is confirmed stable, the cleanup PR (2.13) will: (a) delete the old endpoints and `ExplorationOpportunitySummaryModel`, (b) remove feature flag gates, and (c) **rename v2 endpoints to drop the v2 suffix** (e.g., `/opportunitieshandlerv2` → `/opportunitieshandler`) so they don't confuse future developers.

Decision 5: Extending `TranslatableEntityType` enum

Problem: The `TranslatableEntityType` enum in `feconf.py` currently only has `EXPLORATION` and `QUESTION`. The `EntityTranslationsModel` uses this enum to scope stored translations. We need to add `SKILL` so that skill translations can be stored in `EntityTranslationsModel`.

Decision: Add `SKILL = 'skill'` to the `TranslatableEntityType` enum. Topic and story entity types are not added in this project (they are out of scope per the GSoC project description).

Decision 6: What existing translation infrastructure do we NOT need for metadata and skills?

The current exploration translation system has significant state-level infrastructure (state-by-state navigation, `state_names_to_content_id_mapping`, re-translation cascades, in-place editing). The Q1 Core team sheet asks: "What features do we NOT need for these other translations?"

Analysis - features NOT needed for metadata/skill translations:

Feature	Needed for exploration states	Why
State-by-state navigation / <code>state_names_to_content_id_mapping</code>	No	Metadata has 3–5 flat fields; skills have 2. A flat list suffices.
Complex version-conflict resolution / re-translation cascade	No	Metadata and skill fields are independent and change infrequently. The existing <code>mark_translations_as_needing_update</code> mechanism suffices.
Translation editing after acceptance	Deferred	For metadata/skills, submitting a new suggestion is simpler. In-place editing can be added later.

Decision: Use a simplified flat-list interface for metadata/skills. Reuse the `BaseTranslatableObject` → `EntityTranslationsModel` pipeline but not the state-level UI scaffolding.

Decision: Option B - translate worked examples as part of the explanation content.

Rationale: In the current codebase (post-GSoC 2024 cleanup in PRs [#22902](#), [#22883](#)), WorkedExample is an RTE component embedded inside the explanation HTML as `<oppia-noninteractive-workedexample>` tags. There is no separate `worked_examples` field on SkillContents - it was removed in schema migration v4-->v5. Translating the `explanation_content_id` intrinsically translates the worked examples within it. Option A would require reverting the storage restructuring first, which is outside this project's scope.

Risks and mitigations

Potential Risk	Mitigation
Beam backfill produces incorrect data.	Run dry-run audit job first (extending #25086) to validate computed values before writing. Test on backup server before production.
Skill lifecycle missing opportunity hooks.	Hook into <code>topic_services.publish_topic()</code> and skill-topic assignment handlers (mirrors exploration lifecycle).
Feature flag flip causes regression.	Flip to TEST stage first (backup server). Both old/new endpoints coexist during transition for safe rollback.
PR review delays cause milestone slippage.	Keep PRs small (≤ 400 LOC). Use the greenlight period for early PRs. Keep 2-3 PRs in flight simultaneously.

Key Objectives for Each Milestone

Milestone	Key Objective
Milestone 1	Migrate the existing exploration translation workflow to the generic <code>TranslationOpportunityModel</code> . The contributor dashboard works identically but on the new infrastructure. All tests pass.
Milestone 2	Exploration metadata and skill content can be translated and displayed to learners. Entity-type filtering works. Legacy code cleanup is included as the final PR. Infrastructure is extensible for future entity types.

Implementation Approach

[Web only] Storage Model Layer Changes

TranslationOpportunityModel (already partially implemented at `gae_models.py`, line 268): No schema changes needed. I will add: (a) `get_by_entity_type_and_topic()` - paginated query method filtered by `entity_type` and `topic_ids`, mirroring `ExplorationOpportunitySummaryModel.get_by_topic`, and (b) validation that `translation_counts` keys are valid audio language codes.

GeneralSuggestionModel (existing): No schema changes or migrations are needed to support skill suggestions. The `GeneralSuggestionModel.target_type` field already includes `'skill'` in its choices (`gae_models.py:180`). We will simply reuse it with `target_type="skill"`, updating only the controller validation logic (e.g., in `SuggestionHandler/SuggestionActionHandler`) to accept and process it for `suggestion_type='translate_content'`.

PinnedOpportunityModel modification: Add `entity_type` field (default `'exploration'`) so pinned opportunities can distinguish between entity types.

EntityTranslationsModel - extend enum: Add SKILL = 'skill' to TranslatableEntityType in feconf.py.

[Web only] Storage Model Migrations

Migration 1 - Backfill TranslationOpportunityModel (Beam job): For each ExplorationOpportunitySummaryModel: recomputes content_count from the Exploration's get_translatable_contents_collection() and translation_counts from EntityTranslationsModel (per Decision 3 - recompute, never copy). An audit job runs first to validate computed values.

Migration 2 - Backfill PinnedOpportunityModel (Beam job): Sets entity_type = 'exploration' for all existing pinned opportunities. Simple migration, no computation.

Migration 3 - Create skill translation opportunities (Beam job: CreateSkillTranslationOpportunitiesJob): Reads all published topics, collects linked skill_ids, fetches each Skill, computes content_count via get_translatable_contents_collection(), creates TranslationOpportunityModel with entity_type='skill' and topic_ids listing all topics referencing the skill.

Domain Objects

Extend Exploration.get_translatable_contents_collection(): Add metadata fields before the existing state iteration: exploration_title (unicode), exploration_objective (unicode), and tag_{tagName} for each tag (unicode), all with content_type=ContentType.METADATA. This automatically makes get_content_count() and get_translation_count() include metadata. (See Appendix A for the implementation sketch.)

Make Skill extend BaseTranslatableObject: Implement get_translatable_contents_collection() with skill_description (synthetic, unicode), the explanation's existing content_id (html), and misconception_{id}_feedback for each misconception's feedback. Content count = 2 + N. (See Appendix A for the implementation sketch.)

Extend TranslatableEntityType enum: Add SKILL = 'skill' in feconf.py.

Extend ContentType enum: Add METADATA = 'metadata' in translation_domain.py.

Add TranslationOpportunity methods: increment_translation_count(language_code, new_count) - sets count from recomputed value (not increment-by-1). update_content_count(new_count) - updates and recomputes incomplete_translation_language_codes.

User Flows (Controllers and Services)

Flow 1: Fetching translation opportunities (Contributor)

Current: Frontend --> GET /opportunitiessummaryhandler --> opportunity_services.get_translation_opportunities() --> ExplorationOpportunitySummaryModel.get_by_topic().

New (behind feature flag): Frontend --> GET /opportunitieshandlerv2?entity_type=exploration&language_code=hi&topic_name=Addition --> ContributionOpportunitiesHandlerV2.get() --> opportunity_services.get_translation_opportunities_with_new_models(entity_type, language_code,

topic_name, cursor) --> TranslationOpportunityModel.get_by_entity_type_and_topic() --> returns TranslationOpportunityCardInfo dicts. The new handler is open_access, accepts optional entity_type, required language_code, optional topic_name and cursor. The service method resolves topic_name to topic_id, queries the model, then enriches each result with entity descriptions and topic names. (See Appendix B for handler/service sketches.)

Flow 1 Request:

Parameter	Type	Required	Description
suggestion_type	str	Yes	Always "translate_content"
language_code	str	Yes	ISO 639-1 code (e.g., "hi", "es")
entity_type	str	No	"exploration" "skill" None (returns all)
topic_name	str	No	Filter by topic name
cursor	str	No	Pagination cursor from previous response

Flow 1 Response:

Field	Type	Description
opportunities	List[TranslationOpportunityCardInfoDict]	See below
next_cursor	str None	Cursor for next page
more	bool	True if additional results exist

TranslationOpportunityCardInfoDict fields (verified from opportunity_domain.py:548-560):

Field	Type	Description
entity_type	str	"exploration" or "skill"
entity_id	str	e.g., "exp_123" or "skill_456"
topic_ids	List[str]	All topics this entity belongs to

topic_name	str	Display name of ONE topic (random if multi-topic, filtered if topic param set)
entity_description	str	Exploration title or skill description
content_count	int	Total translatable content items (for skills: 2 + N misconceptions)
incomplete_translation_language_codes	List[str]	Languages with < 100% translation
translation_counts	Dict[str, int]	Map of language_code --> translated count
is_pinned	bool	Whether opportunity is pinned by user
currently_available_to_learners	bool	Whether entity is published

Flow 2: Fetching translatable content for a specific entity

Current: GET /gettranslatabletexthandler?exp_id=xyz&language_code=hi --> returns `state_names_to_content_id_mapping` (keyed by state name).

New: GET /gettranslatablecontentshandlerv2?entity_type=exploration&entity_id=xyz&language_code=hi --> returns a **flat list** of `TranslatableContent` dicts (with `content_id`, `content_type`, `content_format`, `content_value`). For explorations, metadata items appear first, then state-level content. For skills, the list has 2 items (description + explanation). The `entity_version` is included so the frontend can set `target_version_at_submission` on the suggestion. For backward compatibility, the frontend `TranslateTextBackendApiService` transforms the flat list into the `state_names_to_content_id_mapping` format expected by the existing translation modal.

Flow 2 Request:

Parameter	Type	Required	Description
entity_type	str	Yes	"exploration" "skill"
entity_id	str	Yes	Entity identifier
language_code	str	Yes	Target language for translations

Flow 2 Response:

Field	Type	Description
translatable_contents	List[TranslatableContentDict]	See below
entity_version	int	Current version (for staleness detection)

TranslatableContentDict fields:

Field	Type	Description
content_id	str	e.g., "exploration_title", "explanation", "misconception_feedback_1"
content_format	str	"html" "unicode" "unicode_list"
content_type	str	"title" "objective" "tags" "explanation" "feedback"
content_value	str	Original English content
translation_html	str None	Existing translation if present
needs_update	bool	True if source changed since last translation

Flow 3: Submitting a translation suggestion

Changes: 1. `SuggestionHandler.post()` accepts `target_type = 'skill'` in addition to `'exploration'`.

2. For metadata translations, `state_name` uses sentinel value `'__metadata__'` (the existing change command schema requires `state_name`; this avoids modifying the shared schema - can be revisited post-GSoC per TDD).

3. `create_suggestion()` validates `content_id` against the entity's `get_translatable_contents_collection()` for both target types.

Flow 3 Request:

Field	Type	Required	Description
suggestion_type	str	Yes	Always "translate_content"
target_type	str	Yes	"exploration" "skill" (extended from exploration-only)

target_id	str	Yes	Entity identifier
target_version_at_submission	int	Yes	Version when contributor started translating
change_cmd.cmd	str	Yes	Always "add_written_translation"
change_cmd.language_code	str	Yes	Target language
change_cmd.content_id	str	Yes	Content being translated
change_cmd.translation_html	str	Yes	The translated content
change_cmd.data_format	str	Yes	"html" "unicode"

Flow 3 Response:

Field	Type	Description
suggestion_id	str	ID of created suggestion
status	str	"pending" (or "accepted" if auto-accept enabled)

Flow 4: Accepting a translation suggestion

`SuggestionTranslateContent.accept()` dispatches based on `target_type` (`exploration` --> `EXPLORATION`, `skill` --> `SKILL`). `update_translation_opportunity_with_accepted_suggestion()` is generalized to accept (`entity_type`, `entity_id`, `language_code`) and recompute counts from `EntityTranslationsModel` per Decision 3. (See Appendix B for the sketch.)

Flow 4 Request:

Field	Type	Required	Description
action	str	Yes	"accept" "reject" "edit_and_accept"
review_message	str	No	Feedback for contributor
commit_message	str	Yes (for accept)	Commit message
updated_translation_html	str	Yes (for edit_and_accept)	Reviewer's corrected translation

Flow 4 Response:

Field	Type	Description
suggestion_id	str	ID of processed suggestion
status	str	"accepted" "rejected"

Side effects on accept:

- Translation written to `EntityTranslationsModel`
- `TranslationOpportunityModel.translation_counts` RECOMPUTED (not incremented - per Decision 3)
- For tags: `add_translated_tags_to_indexed_exploration_summaries()` called

Flow 5: Reviewing translation suggestions

`GetReviewableOpportunitiesHandlerV2` accepts `entity_type` and `language_code`, fetches reviewable target IDs via `suggestion_services` (generalized from hardcoded exploration), and returns opportunities with pinned items on top.

Flow 5 Request:

Parameter	Type	Required	Description
entity_type	str	Yes	"exploration" "skill"
entity_id	str	Yes	Entity identifier
language_code	str	Yes	Review language
offset	int	No	Pagination offset (default: 0)
limit	int	No	Items per page (default: 20)

Flow 5 Response:

Field	Type	Description
suggestions	List[SuggestionDict]	Pending translation suggestions
target_entity_html	Dict[str, str]	Map of content_id --> current English HTML

Flow 6: Displaying translated content to learners

Three learner-facing endpoints are updated:

- (a) `LibraryIndexHandler` and `ExplorationSummariesHandler` look up `EntityTranslationsModel` to serve translated titles/objectives/tags, falling back to English.
- (b) `ConceptCardDataHandler` accepts optional `language_code` to serve translated skill content (deviation from TDD - endpoint was omitted but needed for the concept card User Story).
- (c) `search_services.index_exploration_summaries()` appends translated tags to the search index.

Flow 7: Opportunity lifecycle for skills

Opportunities are created when a skill is linked to a published topic, deleted when removed from all published topics (with auto-rejection of pending suggestions), and `content_count` is recomputed when skill content is edited. Hook points: `topic_services.publish_topic()` and skill-topic assignment handlers.

Opportunity Model Lifecycle (Detailed)

The `TranslationOpportunityModel` lifecycle is managed through three generalized service functions, extending the current exploration-only pattern in `opportunity_services.py`:

1. CREATE: `add_new_translation_opportunities(entity_types_and_ids: Dict[str, List[str]])`

Trigger Event	Hook Location	Entities Created
Topic published	<code>topic_services.publish_topic()</code>	Topic + all linked skills via <code>topic.uncategorized_skill_ids</code> + <code>subtopic.skill_ids</code>
Story published	<code>topic_services.publish_story()</code>	Story + all linked explorations via <code>story.story_contents.get_all_linked_exp_ids()</code>
Skill saved (if linked to published topic)	<code>skill_services.save_new_skill()</code>	Skill (deferred if no published topic linkage)
Admin regeneration	<code>opportunity_services.regenerate_opportunities_related_to_topic()</code>	All entities in topic

Implementation (extending existing `create_exp_opportunity_summary` pattern at line 201):

- Fetch entities via `get_multiple*_by_id()`
- Compute `topic_ids` via `_compute_topic_ids_of_translation_opportunities()`
- Compute `content_count` via `entity.get_translatable_contents_collection().content_count`
- Compute `translation_counts` by querying `EntityTranslationsModel` for each supported language
- Create `TranslationOpportunityModel` with ID format `{entity_type}.{entity_id}`

2. UPDATE: `compute_translation_opportunity_model_for_updated_entity(entity_type, entity_id, content_count, translation_count)`

Trigger Event	Hook Location
Exploration edited	exp_services.compute_models_to_put_when_saving_new_exp_version()
Exploration reverted	exp_services.revert_exploration()
Skill edited	skill_services.apply_change_list()
Topic edited	topic_services.apply_change_list()
Story edited	story_services.update_story()

Note: The current `update_translation_opportunity_with_accepted_suggestion()` (line 430) uses INCREMENT-BY-1. Per Decision 3 and my PR #24589, the new function will RECOMPUTE `translation_counts` from `EntityTranslationsModel` to avoid count drift.

3. DELETE: `delete_translation_opportunities(entity_types_and_ids: Dict[str, List[str]])`

Trigger Event	Hook Location	Auto-reject Pending Suggestions?
Exploration deleted	exp_services.delete_exploration()	YES (PR 1.6 adds this - current code only deletes model)
Story unpublished	topic_services.unpublish_story()	YES (existing via <code>auto_reject_translation_suggestions_for_exp_ids</code>)
Story deleted	story_services.delete_story()	YES (existing)
Topic unpublished	topic_services.unpublish_topic()	YES
Topic deleted	topic_services.delete_topic()	YES
Skill deleted	skill_services.delete_skill()	YES (existing via <code>auto_reject_question_suggestions_for_skill_id</code>)

Unified Rule: Any entity losing its published path --> delete opportunity model + auto-reject all pending translation suggestions for that entity.

[Web only] Web frontend changes

Learner-Facing Component Updates:

- **ConceptCardComponent**

(`core/templates/components/concept-card/concept-card.component.ts`):

- Add `language_code` prop (derived from the user's site language preference).
- Pass `language_code` to the `/concept_card_handler` API call.
- Display translated `skill_description` and `explanation.html`.
- Falls back to English for missing translations.
- Implemented in PR 2.8

(**Note:** A major advantage of this design is that NO other significant learner-facing frontend component changes are needed to display translated metadata. Existing learner-facing components like `LessonInformationCardModal`, and `LibraryIndex` already render data from JSON returned by backend handlers. By updating the backend handlers to populate the JSON with translated strings, the existing frontend components will naturally display the translations without modification. The remaining frontend changes are in the Contributor Dashboard for submitting/reviewing.)

Contributor Dashboard Component Updates:

Entity Type Selector Component: New `oppia-entity-type-selector` dropdown on both Translate and Review tabs. Options: "All", "Exploration", "Skill". Uses `mat-select` with ARIA label and keyboard navigation. Three new i18n keys: `I18N_ALL_ENTITY_TYPES`, `I18N_ENTITY_TYPE_EXPLORATION`, `I18N_ENTITY_TYPE_SKILL`.

TranslationOpportunitiesComponent updates: Adds `selectedEntityType` property bound to the selector. When it changes, it calls the backend with the new `entity_type` parameter. For skill opportunities, displays skill description instead of chapter/story context. Progress calculation unchanged ($\text{translationCount} / \text{contentCount} * 100$)

TranslationModalComponent updates: Accepts generic `entityType` and `entityId` inputs. For skill entities, fetches from `/gettranslatablecontentshandlerv2?entity_type=skill`. Displays metadata items (title, objective, tags) before state content. Uses correct `target_type` in submit payload.

Backend API Service updates: `ContributionOpportunitiesBackendApiService` adds methods calling new endpoints with `entity_type` param, with feature-flag-aware routing. `TranslateTextBackendApiService` transforms the flat response list into `state_names_to_content_id_mapping` for backward compatibility. `ContributionAndReviewBackendApiService` passes `entity_type` to review endpoints.

Concept Card Component updates: Passes learner's site language to `ConceptCardDataHandler`. Displays translated explanation/description if available; falls back to English.

Searching in User's Preferred Language (PR 2.9)

Current Behavior

The Community Library search uses Elasticsearch index `SEARCH_INDEX_EXPLORATIONS` (defined in `search_services.py:46`). The `index_exploration_summaries()` function indexes explorations with these fields:

`id`, `language_code`, `title`, `category`, `tags`, `objective`, `rank`

Currently, the `tags` field only contains English tag values from `exp_summary.tags`.

Implementation Approach (per TDD lines 945-964)

Two new methods in `search_services.py`:

- **`add_translated_tags_to_indexed_exploration_summaries(translated_text, exploration_id)`** — Called when a tag translation is ACCEPTED. Fetches the indexed document, appends the translated tag to the `tags` list, and re-indexes.
- **`remove_translated_tags_from_indexed_exploration_summaries(translated_text, exploration_id)`** — Called when a translation becomes stale (source content edited). Removes the translated tag from the indexed `tags` list. (See Appendix C for implementation sketches.)

Query Behavior

No changes to `search_explorations()` needed. Elasticsearch naturally matches translated tags since they're now part of the indexed `tags` field.

Example

- **Exploration "Algebra Basics" has English tags:** `["algebra", "math"]`
- **Hindi translation accepted:** "बीजगणित" for "algebra"
- **Updated index:** `tags: ["algebra", "math", "बीजगणित"]`
- **User searches "बीजगणित"** → Elasticsearch matches → "Algebra Basics" appears in results

Edge Cases

- **Tag renamed in English:** Old translation remains in index until Beam job regeneration (PR 2.10)
- **Tag removed:** Corresponding translation removed via `remove_translated_tags_from_indexed_exploration_summaries()`
- **Multiple translations:** Each language's translation appended independently (e.g., `["algebra", "बीजगणित", "álgebra"]`)

Documentation changes

1. Wiki page: Step-by-step guide for adding a new translatable entity type (extend `BaseTranslatableObject`, implement `get_translatable_contents_collection()`, add to enum, create lifecycle hooks, update frontend, write Beam job).
2. Code comments: Comprehensive docstrings on all new service methods explaining opportunity lifecycle, count computation, and entity-type dispatch.

Metrics Plan

Event (see PRD)	Event parameters (see PRD)	Do we already record the event + parameters? - If so, please link to the corresponding code on GitHub. - If not, describe the changes needed to do so.
Translation suggestion submitted	entity_type, entity_id, content_id, language_code	Partially recorded in <code>GeneralSuggestionModel</code> . Add logging for new entity types in community contribution stats update.
Suggestion accepted/rejected	entity_type, language_code, action	Already tracked in <code>GeneralSuggestionModel.status</code> and <code>CommunityContributionStatsModel</code> . No changes needed.
Translation opportunity progress	entity_type, entity_id, language_code, completion_%	Stored in <code>TranslationOpportunityModel.translation_counts</code> . Computable as <code>translation_counts[lang] / content_count * 100</code> .
Translated content displayed to learner	entity_type, entity_id, language_code	Not tracked. Add optional analytics logging in <code>LibraryIndexHandler</code> and <code>ConceptCardDataHandler</code> . Low priority - deferrable.

Implementation Plan

Milestone Table (include both PRs and other actions that need to be taken prior to launch)

Timeline Rationale: PR merge times account for:

- **Review cycles:** 2-3 review rounds (initial feedback, revision, final approval) = 5-7 days typical.
- **Parallel work:** Multiple PRs in flight simultaneously to avoid serial blocking.
- **Holiday buffer:** Jul 11-17 is a holiday week with reduced hours, affecting PRs 2.1 and 2.2.
- **Exam period:** My pre-coding period overlaps with my university final exams (ending May 25), limiting daily hours to 2-3 during the early PR timelines.
- **testing:** Complex PRs (1.9 Beam job, 2.11 acceptance tests) need extra validation time.

Mitigation: Always have 2-3 PRs in review simultaneously. Start the next PR while the current one is being reviewed.

Milestone 1

Key objective for this milestone: Build the extensible translation infrastructure and migrate the translations currently supported by the contributor dashboard (i.e., exploration non-metadata content) to use it. The contributor dashboard works exactly like it did before, but is now implemented using TranslationOpportunityModel. All applicable tests pass robustly.

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1.1	Extend TranslationOpportunityModel (Storage): • Add paginated <code>get_by_entity_type_and_topic()</code> (1 day) • Add batch retrieval and language validation (1 day) • Write unit tests for new methods (1 day) • PR review and revisions (4 days)	-	May 10	May 21
1.2	Extend TranslationOpportunity domain objects: • Add count and topic update methods (1 day) • Add <code>entity_type</code> field to <code>PinnedOpportunity</code> (1 day) • Write domain-level unit tests (1 day) • PR review and revisions (4 days)	-	May 12	May 25
1.3	Implement new opportunity services (Core): • Implement core CRUD methods (reviving closed PR) (1 day) • Implement paginated fetch with enrichment logic (1 day) • Implement recompute-from-source count logic (2 days) • Write comprehensive service-layer unit tests (2 days) • PR review and revisions (4 days)	1.1, 1.2	May 23	May 30
1.4	Implement new controller endpoints: • Implement opportunities and translatable contents handlers (1 day)	1.3	May 31	Jun 7

	- Implement reviewable opportunities and suggestions handlers (1 day) - Gate endpoints behind feature flag and add tests (2 days) - PR review and revisions (3 days)			
1.5	Update uggestionHandler Logic: - Update handlers to resolve against generic models (1 day) - Add feature flag gates for submission workflow (1 day) - Write backend integration tests (1 day) - PR review and revisions (4 days)	1.3	Jun 6	Jun 13
1.6	Update frontend services: - Update ContributionOpportunitiesService routing (1 day) - Update 3 backend API services (opportunities, translate-text, contribution-and-review) (1 day) - Write frontend and backend unit tests (1 day) - PR review and revisions (3 days)	1.4	Jun 8	Jun 14
1.7	Update pinning handler for multi-entity support: - Update pinning handler logic for entity_type (1 day) - Write backend tests for mixed-entity pinning (1 day) - PR review and revisions (4 days)	1.3	Jun 10	Jun 16
1.8	PM demo (week 4): Prepare demo for opportunity listing, review, and pinning (1 day)	1.5, 1.6, 1.7	Jun 21	-
1.9	Beam job to backfill TranslationOpportunityModel: - Design Beam pipeline for multi-entity backfill (1 day) - Implement PTransform for count recomputation (1 day) - Add audit output and error handling (1 day)	1.1, 1.2	May 24	Jun 8

	- Write unit tests and testing doc (2 days) - PR review and revisions (5 days)			
1.10	Beam job to backfill PinnedOpportunityModel: - Write Beam job to backfill entity_type field (1 day) - Write unit tests and testing doc (1 day) - PR review and revisions (4 days)	1.2	Jun 13	Jun 19
1.11	Acceptance tests for migrated workflow: - Write tests for opportunity listing and submission (1 day) - Write tests for review and progress updates (1 day) - Fix flakiness and run locally (1 day) - PR review and revisions (5 days)	1.5, 1.6	Jun 16	Jun 24
-	Jun 26: Milestone 1 work due for internal evaluation. All PRs submitted by this date.			
-	Jun 27 – Jul 3: Testing of Milestone 1 work product.			
	Jul 4 – Jul 10: Buffer time for Milestone 1 revisions.			

Milestone 2

Key objective for this milestone: Exploration metadata and skill content can be translated through the contributor dashboard and displayed to learners. Translators and reviewers can filter by entity type and topic. All applicable tests pass robustly. Legacy code cleanup is included in final PR.

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge

2.1	Add METADATA content type and domain logic: • Add <code>ContentType.METADATA</code> to exploration domain (1 day) • Update <code>Exploration.get_translatable_contents_collection()</code> (1 day) • Write unit tests for metadata extraction (1 day) • PR review and revisions (4 days)	Milestone 1 complete	Jul 11	Jul 18
2.2	Make Skill extend BaseTranslatableObject: • Implement <code>get_translatable_contents_collection()</code> for Skills (1 day) • Add synthetic IDs for misconception feedback (1 day) • Write domain tests for skill translatable content (1 day) • PR review and revisions (5 days)	-	Jul 14	Jul 21
2.3	Implement skill opportunity lifecycle in services: • Add hooks in <code>topic_services</code> for skill opportunity updates (1 day) • Add auto-rejection of suggestions on skill removal (1 day) • Write service-layer lifecycle unit tests (1 day) • PR review and revisions (4 days)	2.2, M1 1.3	Jul 21	Jul 28
2.4	Update controllers for skill translations: • Extend <code>SuggestionHandler</code> for <code>target_type = 'skill'</code> (1 day) • Update accept logic for <code>SKILL</code> type translations (1 day) • Write backend integration tests (1 day) • PR review and revisions (4 days)	2.2, 2.3	Jul 24	Jul 31
2.5	Add entity-type-selector frontend component: • Create <code>EntityTypeSelectorComponent</code> (1 day) • Wire into Translate and Review tabs with i18n keys (1 day) • Update <code>TranslationModalComponent</code> for skill entities (1 day)	2.4, M1 1.5	Jul 28	Aug 4

	Write frontend tests and PR review (4 days)			
2.6	Beam job to backfill skill opportunities: - Collect skills from published topics and create models (2 days) - Write tests and testing doc (2 days) - PR review and revisions (4 days)	2.2, 2.3	Jul 31	Aug 7
2.7	Display translated exploration metadata: - Update <code>LibraryIndexHandler</code> and <code>ExplorationSummariesHandler</code> (1 day) - Write backend/frontend tests for metadata display (2 days) - PR review and revisions (4 days)	2.1	Aug 4	Aug 11
2.8	Display translated concept cards: - Update <code>ConceptCardDataHandler</code> for language_code support (1 day) - Implement fallback to English for non-translated cards (1 day) - Write unit and integration tests (1 day) - PR review and revisions (4 days)	2.2, 2.5	Aug 7	Aug 14
2.9	Search with translated tags: - Extend search index with <code>EntityTranslationsModel</code> tags (2 days) - Write backend tests for multi-language search (1 day) - PR review and revisions (4 days)	2.1, 2.7	Aug 9	Aug 15
2.10	PM DEMO (Week 5): Prepare demo for metadata/skill translation and search (1 day)	2.5, 2.8	Aug 11	-
2.11	Acceptance tests for multi-entity translation:	2.7, 2.8	Aug 14	Aug 19

	- Write tests for metadata and skill translation flows (1 day) - Write tests for learner-facing translated display (1 day) - PR review and revisions (3 days)			
2.12	Wiki page: Adding new translatable entity types: - Write step-by-step technical guide (2 days) - Review and revisions (3 days)	Architecture	Aug 14	Aug 19
2.13	Remove legacy code and feature cleanup: - Delete ExplorationOpportunitySummaryModel (1 day) - Drop v2 suffixes and remove feature flag gates (1 day) - PR merge and final validation (1 day)	2.11	Aug 18	Aug 21
-	Aug 19: Milestone 2 work due for internal evaluation. All PRs submitted by this date.	-	-	-
-	Aug 20 – Aug 30: Testing of Milestone 2 work product.	-	-	-
-	Aug 31 – Sep 2: Buffer time for Milestone 2 revisions.	-	-	-

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Need sync between Android / Web?	Key stakeholders
1.	ENABLE_TRANSLATION_OPPORTUNITIES_WITH_NEW_OPP_MODELS (existing, currently DEV stage)	The contributor dashboard uses TranslationOpportunityModel instead of ExplorationOpportunitySummaryModel for all translation opportunity operations. v2 endpoints (/opportunitieshandlerv2 , etc.) become active. The entity-type selector appears in the UI. Skills and exploration metadata become available for translation.	No (web only)	@chris7716, @masterboy376

2.	(No additional flags needed)	All changes in this project are gated behind the single existing flag above. This simplifies the rollout and avoids the complexity of multiple flags.	-	-
----	------------------------------	---	---	---

Future Work

Note: This section is mainly for reference (since items listed here won't be part of the GSoC project). Proposals will primarily be evaluated based on the implementation plan above.

- Topic/Story metadata translations: Extend [Topic](#) and [Story](#) as [BaseTranslatableObjects](#).
- Misconception/rubric translations for skills: Could use the exact same synthetic content ID pattern (e.g., [misconception_{id}_name](#)) developed in this project, deferred to future work alongside rubrics.
- Translation count drift monitoring: Extend audit Beam job to validate all entity types periodically.

Appendix A: Domain Object Implementation Sketches

These sketches illustrate the key domain changes and show exact files/methods affected by each PR. For reference only - may evolve during implementation.

Exploration.get_translatable_contents_collection() - metadata extension:

```
def get_translatable_contents_collection(self, **kwargs):
    collection = TranslatableContentsCollection()

    # NEW: Add metadata fields
    collection.add_translatable_field(
        content_id='exploration_title',
        content_type=ContentType.METADATA,
        content_format=TranslatableContentFormat.UNICODE_STRING,
        content_value=self.title)
    collection.add_translatable_field(
        content_id='exploration_objective',
        content_type=ContentType.METADATA,
        content_format=TranslatableContentFormat.UNICODE_STRING,
        content_value=self.objective)
    for tag in self.tags:
        collection.add_translatable_field(
            content_id=f'tag_{tag}',
            content_type=ContentType.METADATA,
            content_format=TranslatableContentFormat.UNICODE_STRING,
            content_value=tag)

    # EXISTING: Iterate over states
    ...
```

Skill.get_translatable_contents_collection():

```
def get_translatable_contents_collection(self, **kwargs):
    collection = TranslatableContentsCollection()

    # 1. Skill Description
    collection.add_translatable_field(
        content_id='skill_description',
        content_type=ContentType.METADATA,
        content_format=TranslatableContentFormat.UNICODE_STRING,
        content_value=self.description)

    # 2. Explanation (includes Worked Examples)
    collection.add_translatable_field(
        content_id=self.skill_contents.explanation.content_id,
        content_type=ContentType.CONTENT,
        content_format=TranslatableContentFormat.HTML,
        content_value=self.skill_contents.explanation.html)

    # 3. Misconception Feedback (Dynamically generated N times)
    for misconception in self.misconceptions:
        collection.add_translatable_field(
            content_id=f'misconception_{misconception.id}_feedback',
            content_type=ContentType.CONTENT,
            content_format=TranslatableContentFormat.HTML,
            content_value=misconception.feedback)

    return collection
```

Appendix B: Controller and Service Implementation Sketches

These sketches illustrate the key handler and service logic. For reference only.

update_translation_opportunity_with_accepted_suggestion() - recompute logic:

```
def update_translation_opportunity_with_accepted_suggestion(
    entity_type, entity_id, language_code
):
    opp_model = TranslationOpportunityModel.get(
        TranslationOpportunityModel._generate_id(
            entity_type, entity_id))
```

```

# Recompute from source of truth (same pattern as PR #24589).
entity = _fetch_entity(entity_type, entity_id)
current_translation_counts = (
    translation_services.get_translation_counts(
        feconf.TranslatableEntityType(entity_type), entity))
if language_code in current_translation_counts:
    opp_model.translation_counts[language_code] = (
        current_translation_counts[language_code])
else:
    opp_model.translation_counts[language_code] = 0
# Update incomplete_translation_language_codes accordingly.
if (opp_model.translation_counts[language_code]
    >= opp_model.content_count):
    if language_code in opp_model.incomplete_translation_language_codes:
        opp_model.incomplete_translation_language_codes.remove(
            language_code)
    else:
        if language_code not in opp_model.incomplete_translation_language_codes:
            opp_model.incomplete_translation_language_codes.append(
                language_code)
opp_model.update_timestamps()
opp_model.put()

```

ContributionOpportunitiesHandlerV2 - handler sketch:

```

class ContributionOpportunitiesHandlerV2(base.BaseHandler):
    HANDLER_ARGS_SCHEMAS = {
        'GET': {
            'entity_type': {
                'schema': {'type': 'basestring'},
                'default_value': None
            },
            'language_code': {
                'schema': {
                    'type': 'basestring',
                    'validators': [{
                        'id': 'is_supported_audio_language_code'
                    }]
                }
            },
            'topic_name': {
                'schema': {'type': 'basestring'},
                'default_value': None
            },
            'cursor': {

```

```

        'schema': {'type': 'basestring'},
        'default_value': None
    }
}

}

@acl_decorators.open_access
def get(self):
    entity_type = self.normalized_request.get('entity_type')
    language_code = self.normalized_request['language_code']
    topic_name = self.normalized_request.get('topic_name')
    cursor = self.normalized_request.get('cursor')

    opportunities, next_cursor, more = (
        opportunity_services.get_translation_opportunities_with_new_models(
            entity_type, language_code, topic_name, cursor
        )
    )

    self.render_json({
        'opportunities': [opp.to_dict() for opp in opportunities],
        'next_cursor': next_cursor,
        'more': more
    })

```

Appendix C: Search Index Implementation Sketches

`add_translated_tags_to_indexed_exploration_summaries()` (per TDD lines 947-955):

```

def add_translated_tags_to_indexed_exploration_summaries(
    translated_text: str, exploration_id: str
) -> None:
    # 1. Fetch current indexed document
    doc = platform_search_services.get_document(
        exploration_id, SEARCH_INDEX_EXPLORATIONS
    )
    # 2. Append translated tag to existing tags list
    doc['tags'].append(translated_text)
    # 3. Re-index the document
    platform_search_services.add_documents_to_index(
        [doc], SEARCH_INDEX_EXPLORATIONS
    )

```

`remove_translated_tags_from_indexed_exploration_summaries()` (per TDD lines 956-964):

```
def remove_translated_tags_from_indexed_exploration_summaries(
    translated_text: str, exploration_id: str
) -> None:
    doc = platform_search_services.get_document(
        exploration_id, SEARCH_INDEX_EXPLORATIONS
    )
    if translated_text in doc['tags']:
        doc['tags'].remove(translated_text)
    platform_search_services.add_documents_to_index(
        [doc], SEARCH_INDEX_EXPLORATIONS
    )
```